Pengantar Sain Data

Konsep dan Aplikasi untuk Bisnis

To blah, blah, and blah.

Daftar Isi

Selamat Belajar Data **vii**

Selamat Belajar Data **vii**

ix

Charting in Colaboratory **xi**

0.1 Matplotlib xi

0.1.1 Line Plots xi

0.1.2 Bar Plots xii

0.1.3 Histograms xiii

0.1.4 Scatter Plots xvi

0.1.5 Stack Plots xvii

0.1.6 Pie Charts xviii

0.1.7 fill_between and alpha xix

0.1.8 Subplotting using Subplot2grid xx

0.2 Plot styles xxi

0.3 3D Graphs xxi

0.3.1 3D Scatter Plots xxi

0.3.2 3D Bar Plots xxiii

0.3.3 Wireframe Plots xxiv

0.4 Seaborn xxv

0.5 Altair xxvii

0.6 Plotly xxvii

0.6.1 Sample xxvii

0.7 Bokeh xxviii
0.7.1 Sample xxviii

Packages **xxxi**

0.8 Load Dataset xxxii
0.9 Data preparation xxxii
0.10 The first four moments *of the iris statistical distribution* xxxiii

Exploratory data visualisation **xxxv**

0.10.1 Box plot xxxv
0.10.2 Histograms and Kernel Density estimation xxxvi
0.11 Multivariate analysis xxxix
0.11.1 Pairwise comparison xxxix
0.11.2 Scatterplot xli
0.11.3 Pairgrid xliii
0.11.4 Scatter matrix plot xliv
0.11.5 Pairwise plot xlv
0.11.6 Radviz plot xlvi
0.11.7 Andrews curves xlvii
0.11.8 Parallel coordinates xlix
0.11.9 Ground truth - Three dimenstional plot l

K-means clustering **liii**

Optimisation **lv**

0.11.10 Sihlouette method lv
0.11.11 Elbow method lvi

Train model **lix**

Visualise predictions: k-means **lxi**

Contoh **lxiii**

Summary **lxv**

References **lxvii**

References **lxvii**

Selamat Belajar Data

Copyright © 2023.

Second Edition.

Version date: November 25, 2023.

This textbook and its supplements, including slides, labs, and interactive tutorials, may be downloaded for free at
openintro.org/book/ims[1].

This textbook is a derivative of *OpenIntro Statistics* 4th Edition and *Introduction to Statistics with Randomization and Simulation* 1st Edition by Diez, Barr, and Çetinkaya-Rundel, and it's available under a Creative Commons Attribution-ShareAlike 3.0 Unported United States License. License details are available at the Creative Commons website:
creativecommons.org[2].

Source files for this book can be found on GitHub at
github.com/OpenIntroStat/ims[3].

[1] http://openintro.org/book/ims
[2] https://www.openintro.org/go/?id=creativecommons_org&referrer=ims1_pdf
[3] https://github.com/OpenIntroStat/ims

0

Charting in Colaboratory

A common use for notebooks is data visualization using charts. Colaboratory makes this easy with several charting tools available as Python imports.

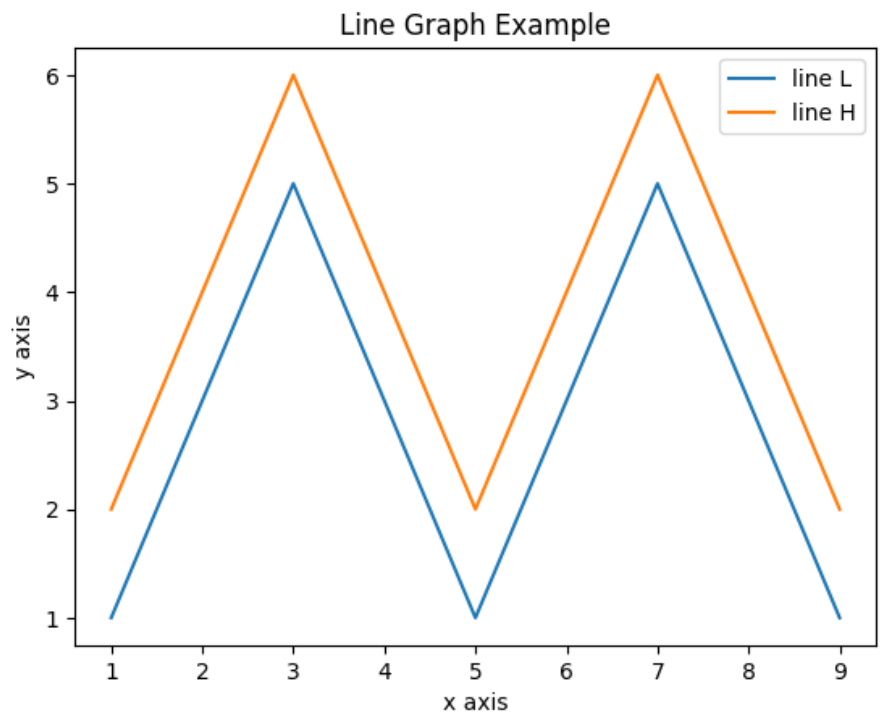
0.1 Matplotlib

Matplotlib[4] is the most common charting package, see its documentation[5] for details, and its examples[6] for inspiration.

0.1.1 Line Plots

```
import matplotlib.pyplot as plt

x  = [1, 2, 3, 4, 5, 6, 7, 8, 9]
y1 = [1, 3, 5, 3, 1, 3, 5, 3, 1]
y2 = [2, 4, 6, 4, 2, 4, 6, 4, 2]
plt.plot(x, y1, label="line L")
plt.plot(x, y2, label="line H")
plt.plot()

plt.xlabel("x axis")
plt.ylabel("y axis")
plt.title("Line Graph Example")
plt.legend()
plt.show()
```

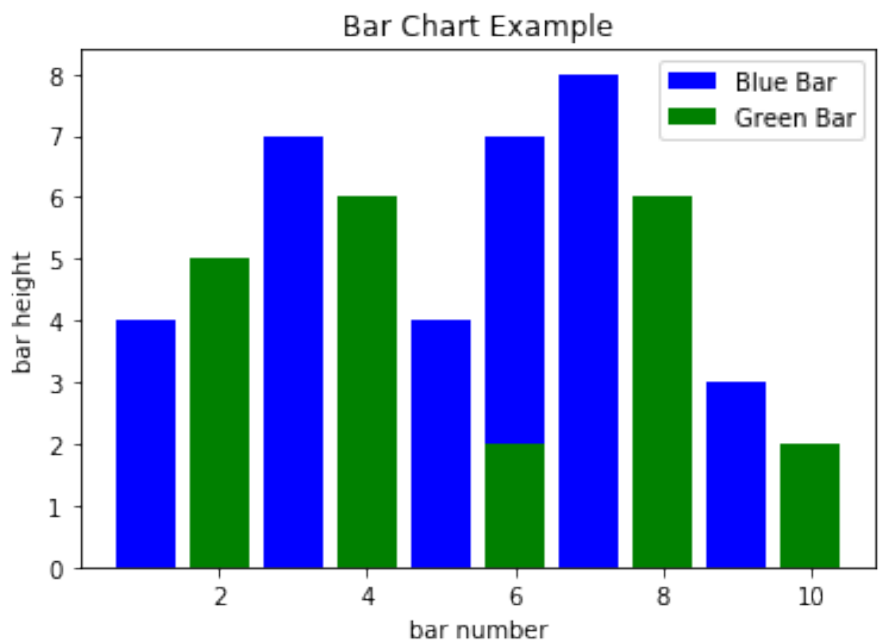
[4] http://matplotlib.org/
[5] http://matplotlib.org/api/pyplot_api.html
[6] http://matplotlib.org/gallery.html#statistics



0.1.2 Bar Plots

```
import matplotlib.pyplot as plt

# Look at index 4 and 6, which demonstrate overlapping cases.
x1 = [1, 3, 4, 5, 6, 7, 9]
y1 = [4, 7, 2, 4, 7, 8, 3]

x2 = [2, 4, 6, 8, 10]
y2 = [5, 6, 2, 6, 2]

# Colors: https://matplotlib.org/api/colors_api.html

plt.bar(x1, y1, label="Blue Bar", color='b')
plt.bar(x2, y2, label="Green Bar", color='g')
plt.plot()

plt.xlabel("bar number")
```

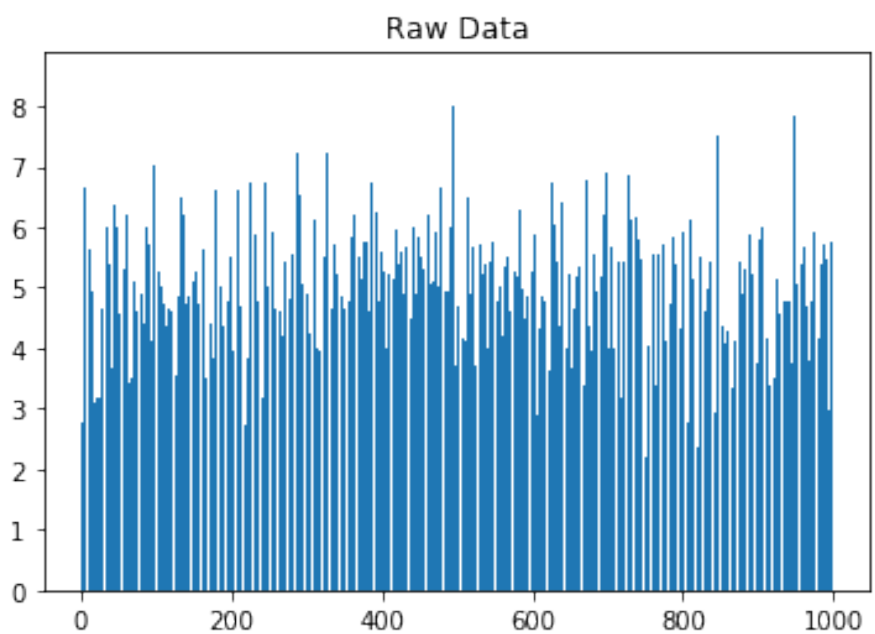
```
plt.ylabel("bar height")
plt.title("Bar Chart Example")
plt.legend()
plt.show()
```

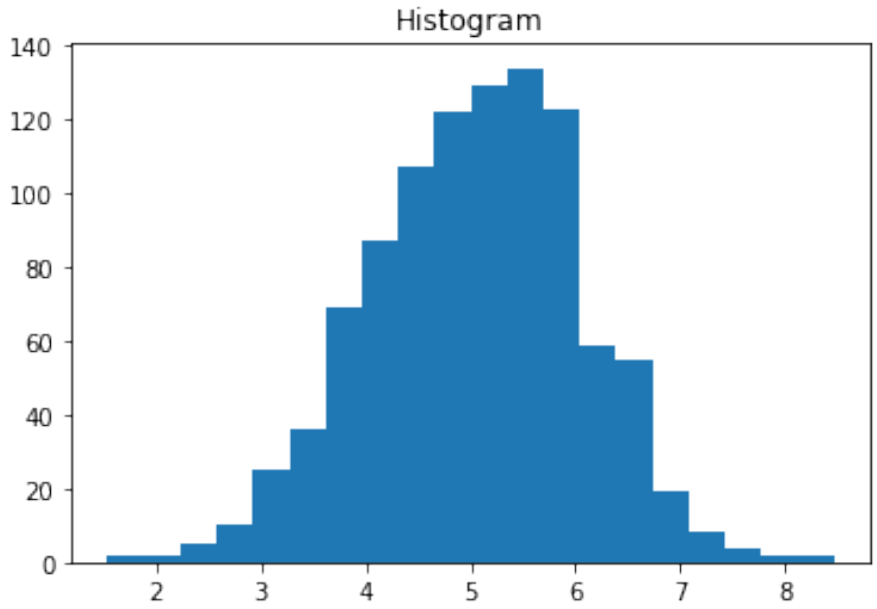


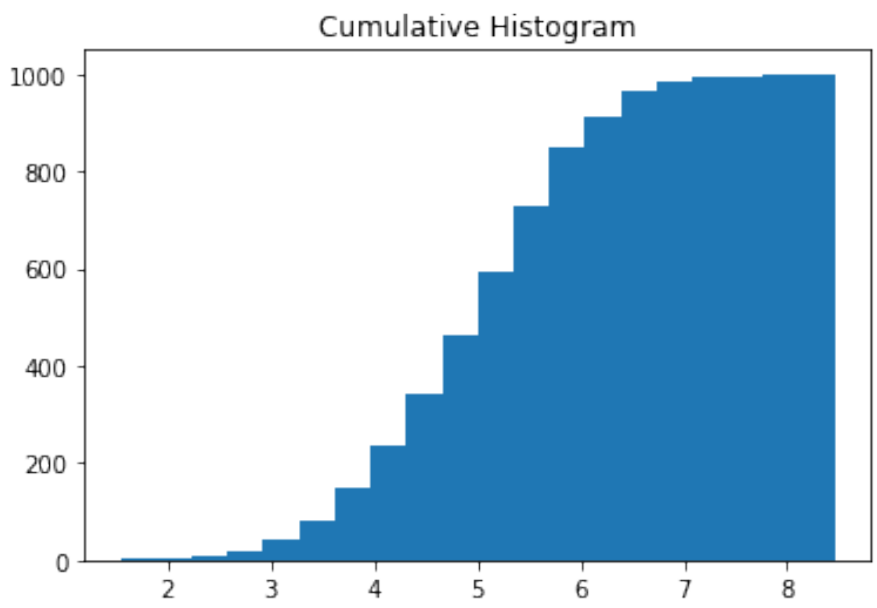
0.1.3 Histograms

```
import matplotlib.pyplot as plt
import numpy as np

# Use numpy to generate a bunch of random data in a bell curve
 ↪  around 5.
n = 5 + np.random.randn(1000)

m = [m for m in range(len(n))]
plt.bar(m, n)
plt.title("Raw Data")
plt.show()
```

```
plt.hist(n, bins=20)
plt.title("Histogram")
plt.show()

plt.hist(n, cumulative=True, bins=20)
plt.title("Cumulative Histogram")
plt.show()
```

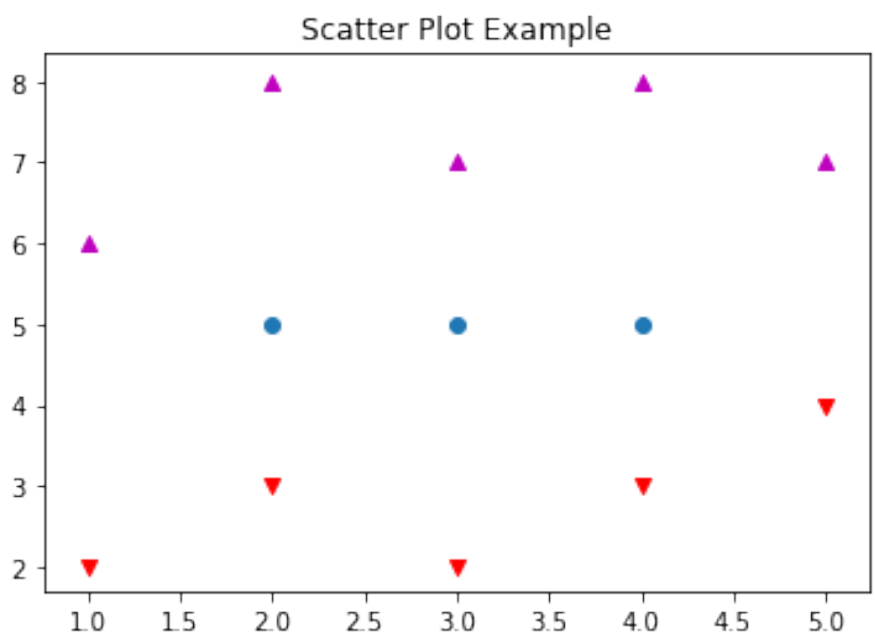



Histogram
140
120
100
80
60
40
20
0
2
3
4
5
6
7
8


Cumulative Histogram
1000
800
600
400
200
0
2
3
4
5
6
7
8

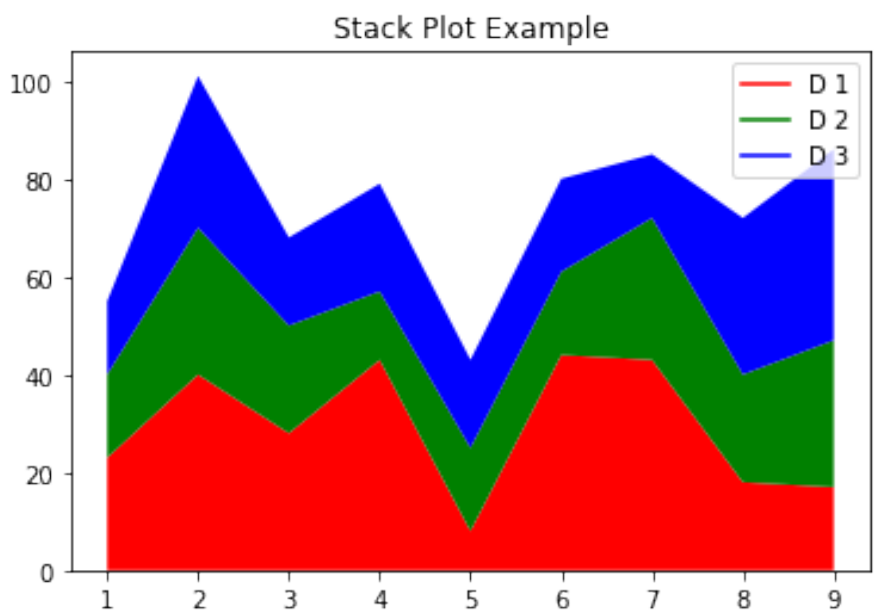
0.1.4 Scatter Plots

```
import matplotlib.pyplot as plt

x1 = [2, 3, 4]
y1 = [5, 5, 5]

x2 = [1, 2, 3, 4, 5]
y2 = [2, 3, 2, 3, 4]
y3 = [6, 8, 7, 8, 7]

# Markers: https://matplotlib.org/api/markers_api.html

plt.scatter(x1, y1)
plt.scatter(x2, y2, marker='v', color='r')
plt.scatter(x2, y3, marker='^', color='m')
plt.title('Scatter Plot Example')
plt.show()
```



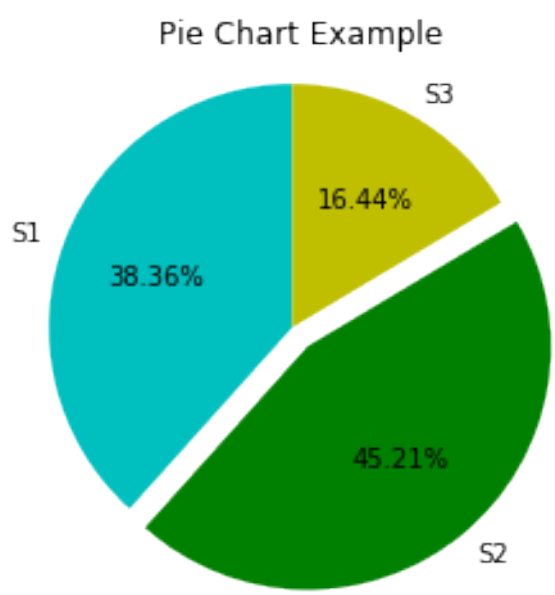
0.1.5 Stack Plots

```
import matplotlib.pyplot as plt

idxes = [ 1,  2,  3,  4,  5,  6,  7,  8,  9]
arr1  = [23, 40, 28, 43,  8, 44, 43, 18, 17]
arr2  = [17, 30, 22, 14, 17, 17, 29, 22, 30]
arr3  = [15, 31, 18, 22, 18, 19, 13, 32, 39]

# Adding legend for stack plots is tricky.
plt.plot([], [], color='r', label = 'D 1')
plt.plot([], [], color='g', label = 'D 2')
plt.plot([], [], color='b', label = 'D 3')

plt.stackplot(idxes, arr1, arr2, arr3, colors= ['r', 'g', 'b'])
plt.title('Stack Plot Example')
plt.legend()
plt.show()
```



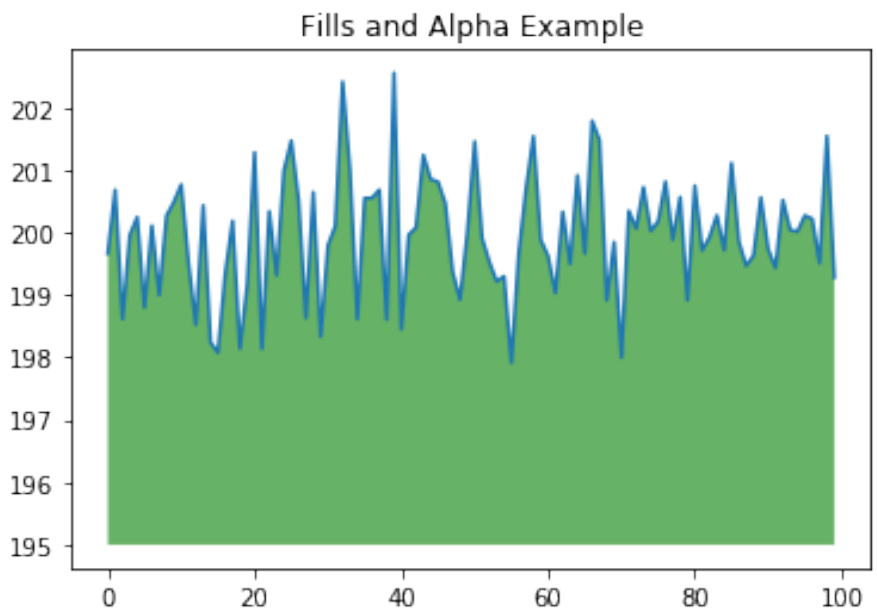
0.1.6 Pie Charts

```
import matplotlib.pyplot as plt

labels = 'S1', 'S2', 'S3'
sections = [56, 66, 24]
colors = ['c', 'g', 'y']

plt.pie(sections, labels=labels, colors=colors,
        startangle=90,
        explode = (0, 0.1, 0),
        autopct = '%1.2f%%')

plt.axis('equal') # Try commenting this out.
plt.title('Pie Chart Example')
plt.show()
```



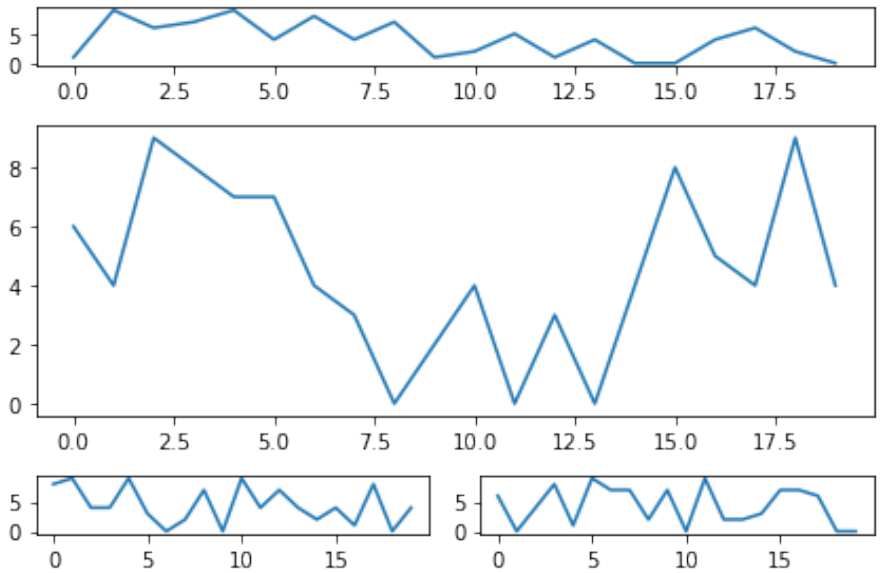
0.1.7 fill_between and alpha

```
import matplotlib.pyplot as plt
import numpy as np

ys = 200 + np.random.randn(100)
x = [x for x in range(len(ys))]

plt.plot(x, ys, '-')
plt.fill_between(x, ys, 195, where=(ys > 195), facecolor='g',
 ↪  alpha=0.6)

plt.title("Fills and Alpha Example")
plt.show()
```



0.1.8 Subplotting using Subplot2grid

```
import matplotlib.pyplot as plt
import numpy as np

def random_plots():
  xs = []
  ys = []

  for i in range(20):
    x = i
    y = np.random.randint(10)

    xs.append(x)
    ys.append(y)

  return xs, ys

fig = plt.figure()
ax1 = plt.subplot2grid((5, 2), (0, 0), rowspan=1, colspan=2)
ax2 = plt.subplot2grid((5, 2), (1, 0), rowspan=3, colspan=2)
ax3 = plt.subplot2grid((5, 2), (4, 0), rowspan=1, colspan=1)
ax4 = plt.subplot2grid((5, 2), (4, 1), rowspan=1, colspan=1)

x, y = random_plots()
ax1.plot(x, y)

x, y = random_plots()
ax2.plot(x, y)

x, y = random_plots()
ax3.plot(x, y)

x, y = random_plots()
ax4.plot(x, y)

plt.tight_layout()
plt.show()
```



0.2 Plot styles

Colaboratory charts use Seaborn's[7] custom styling by default. To customize styling further please see the matplotlib docs[8].

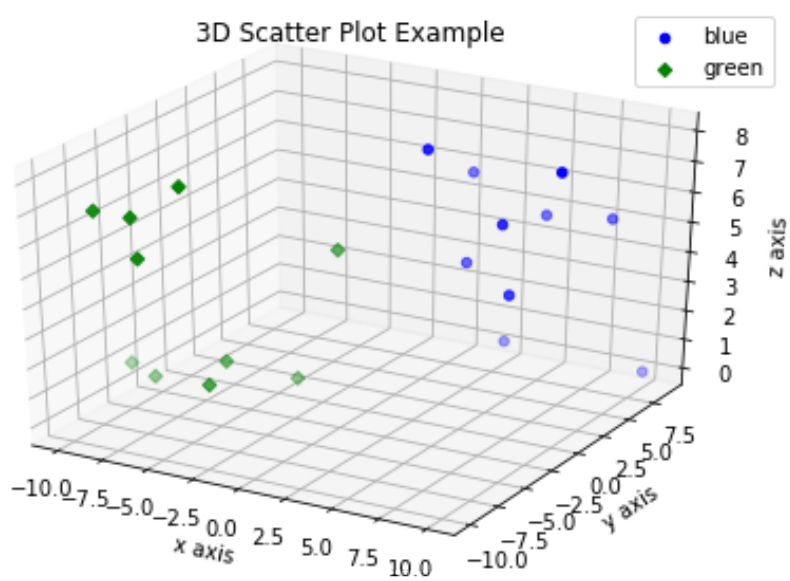
0.3 3D Graphs

0.3.1 3D Scatter Plots

```
import matplotlib.pyplot as plt
import numpy as np
from mpl_toolkits.mplot3d import axes3d

fig = plt.figure()
```

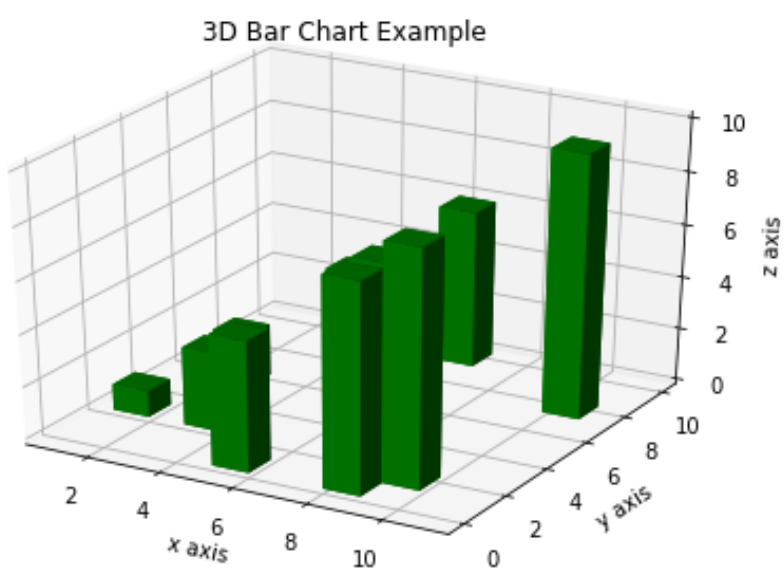
[7] https://seaborn.pydata.org
[8] https://matplotlib.org/users/style_sheets.html

```
ax = fig.add_subplot(111, projection = '3d')

x1 = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
y1 = np.random.randint(10, size=10)
z1 = np.random.randint(10, size=10)

x2 = [-1, -2, -3, -4, -5, -6, -7, -8, -9, -10]
y2 = np.random.randint(-10, 0, size=10)
z2 = np.random.randint(10, size=10)

ax.scatter(x1, y1, z1, c='b', marker='o', label='blue')
ax.scatter(x2, y2, z2, c='g', marker='D', label='green')

ax.set_xlabel('x axis')
ax.set_ylabel('y axis')
ax.set_zlabel('z axis')
plt.title("3D Scatter Plot Example")
plt.legend()
plt.tight_layout()
plt.show()
```



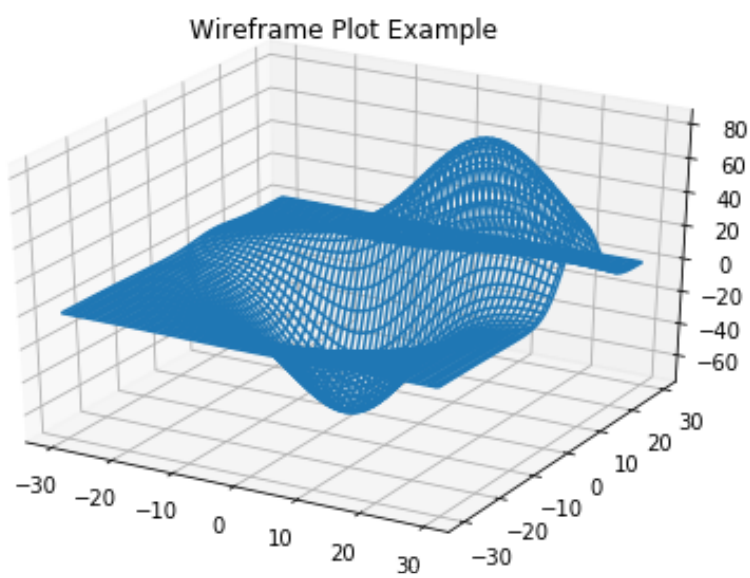
0.3.2 3D Bar Plots

```
import matplotlib.pyplot as plt
import numpy as np

fig = plt.figure()
ax = fig.add_subplot(111, projection = '3d')

x = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
y = np.random.randint(10, size=10)
z = np.zeros(10)

dx = np.ones(10)
dy = np.ones(10)
dz = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

ax.bar3d(x, y, z, dx, dy, dz, color='g')

ax.set_xlabel('x axis')
ax.set_ylabel('y axis')
ax.set_zlabel('z axis')
plt.title("3D Bar Chart Example")
plt.tight_layout()
plt.show()
```



0.3.3 Wireframe Plots

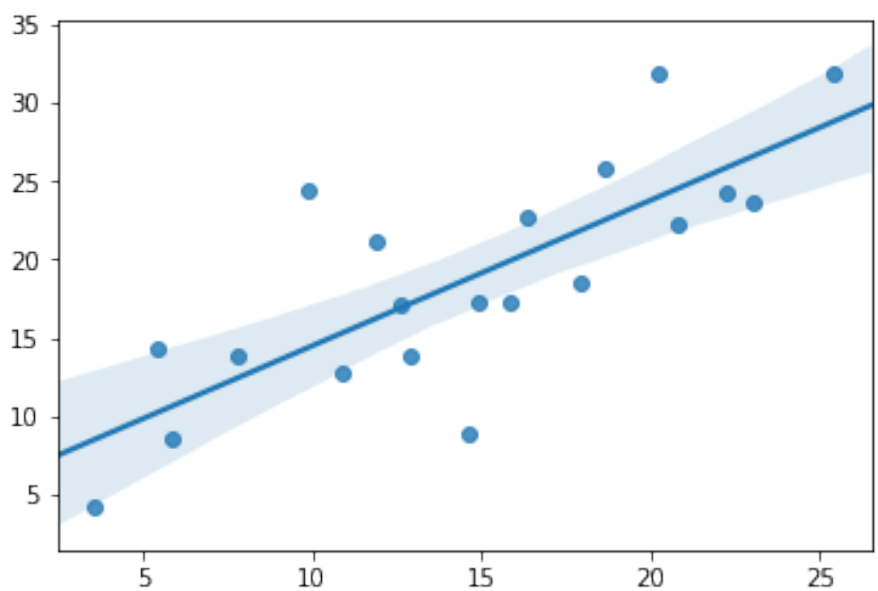
```
import matplotlib.pyplot as plt

fig = plt.figure()
ax = fig.add_subplot(111, projection = '3d')

x, y, z = axes3d.get_test_data()

ax.plot_wireframe(x, y, z, rstride = 2, cstride = 2)

plt.title("Wireframe Plot Example")
plt.tight_layout()
plt.show()
```



0.4 Seaborn

There are several libraries layered on top of Matplotlib that you can use in Colab. One that is worth highlighting is Seaborn[9]:

```
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns

# Generate some random data
num_points = 20
# x will be 5, 6, 7... but also twiddled randomly
x = 5 + np.arange(num_points) + np.random.randn(num_points)
# y will be 10, 11, 12... but twiddled even more randomly
y = 10 + np.arange(num_points) + 5 * np.random.randn(num_points)
sns.regplot(x, y)
plt.show()
```

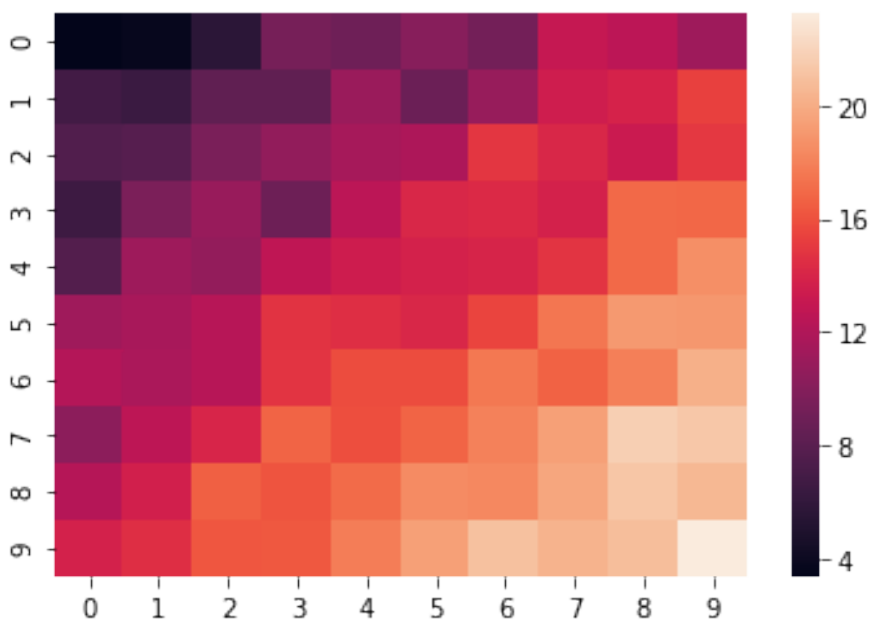


[9] http://seaborn.pydata.org

That's a simple scatterplot with a nice regression line fit to it, all with just one call to Seaborn's regplot[10].

Here's a Seaborn heatmap[11]:

```
import matplotlib.pyplot as plt
import numpy as np

# Make a 10 x 10 heatmap of some random data
side_length = 10
# Start with a 10 x 10 matrix with values randomized around 5
data = 5 + np.random.randn(side_length, side_length)
# The next two lines make the values larger as we get closer to
 ↪  (9, 9)
data += np.arange(side_length)
data += np.reshape(np.arange(side_length), (side_length, 1))
# Generate the heatmap
sns.heatmap(data)
plt.show()
```



[10] http://seaborn.pydata.org/generated/seaborn.regplot.html#seaborn.regplot
[11] https://seaborn.pydata.org/generated/seaborn.heatmap.html

0.5 Altair

Altair[12] is a declarative visualization library for creating interactive visualizations in Python, and is installed and enabled in Colab by default.

For example, here is an interactive scatter plot:

```
import altair as alt
from vega_datasets import data
cars = data.cars()

alt.Chart(cars).mark_point().encode(
    x='Horsepower',
    y='Miles_per_Gallon',
    color='Origin',
).interactive()
```

```
alt.Chart(...)
```

For more examples of Altair plots, see the Altair snippets notebook[13] or the external Altair Example Gallery[14].

0.6 Plotly

0.6.1 Sample

```
from plotly.offline import iplot
import plotly.graph_objs as go

data = [
    go.Contour(
        z=[[10, 10.625, 12.5, 15.625, 20],
           [5.625, 6.25, 8.125, 11.25, 15.625],
```

[12] http://altair-viz.github.io
[13] /notebooks/snippets/altair.ipynb
[14] https://altair-viz.github.io/gallery/

```
            [2.5, 3.125, 5., 8.125, 12.5],
            [0.625, 1.25, 3.125, 6.25, 10.625],
            [0, 0.625, 2.5, 5.625, 10]]
    )
]
iplot(data)
```

```
<IPython.core.display.HTML object>
```

```
Unable to display output for mime type(s): text/html
```

```
Unable to display output for mime type(s): application/vnd.plotly.v1+json, text/html
```

0.7 Bokeh

0.7.1 Sample

```
import numpy as np
from bokeh.plotting import figure, show
from bokeh.io import output_notebook

# Call once to configure Bokeh to display plots inline in the
 ↪ notebook.
output_notebook()
```

```
N = 4000
x = np.random.random(size=N) * 100
y = np.random.random(size=N) * 100
radii = np.random.random(size=N) * 1.5
colors = ["#%02x%02x%02x" % (r, g, 150) for r, g in
 ↪ zip(np.floor(50+2*x).astype(int),
 ↪ np.floor(30+2*y).astype(int))]

p = figure()
p.circle(x, y, radius=radii, fill_color=colors, fill_alpha=0.6,
 ↪ line_color=None)
show(p)
```

```
Unable to display output for mime type(s): text/html
```

0

Packages

```
import sys

import pandas as pd
import numpy as np
import matplotlib
import matplotlib.pyplot as plt
from pandas.plotting import parallel_coordinates,
 ↪  andrews_curves, radviz, scatter_matrix
import seaborn as sns

import sklearn
from scipy import stats

from IPython.display import set_matplotlib_formats
matplotlib.style.use('ggplot') # Other styles
 ↪  matplotlib.style.available
import os
from warnings import filterwarnings
filterwarnings('ignore')

%matplotlib inline
sns.set_context('notebook')
plt.close('all')
```

0.8 Load Dataset

```
url =
 ↪  'https://raw.githubusercontent.com/pandas-dev/pandas/master/pandas/tests/data/iris.csv'
iris = pd.read_csv(url)
iris.rename(columns={'Name':'species'},inplace=True)
```

0.9 Data preparation

```
print(iris.info())
iris.describe(include='all')
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   SepalLength  150 non-null    float64
 1   SepalWidth   150 non-null    float64
 2   PetalLength  150 non-null    float64
 3   PetalWidth   150 non-null    float64
 4   species      150 non-null    object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
None
```

	SepalLength	SepalWidth	PetalLength	PetalWidth	species
count	150.000000	150.000000	150.000000	150.000000	150
unique	NaN	NaN	NaN	NaN	3
top	NaN	NaN	NaN	NaN	Iris-virginica
freq	NaN	NaN	NaN	NaN	50
mean	5.843333	3.054000	3.758667	1.198667	NaN
std	0.828066	0.433594	1.764420	0.763161	NaN
min	4.300000	2.000000	1.000000	0.100000	NaN

	SepalLength	SepalWidth	PetalLength	PetalWidth	species
25%	5.100000	2.800000	1.600000	0.300000	NaN
50%	5.800000	3.000000	4.350000	1.300000	NaN
75%	6.400000	3.300000	5.100000	1.800000	NaN
max	7.900000	4.400000	6.900000	2.500000	NaN

Species is stored as an object, instead of as categorical variable, this will not be ideal since is our target varible. It would be best to store it as a categorical variable

```
iris.species = iris.species.astype('category')
iris.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   SepalLength  150 non-null    float64
 1   SepalWidth   150 non-null    float64
 2   PetalLength  150 non-null    float64
 3   PetalWidth   150 non-null    float64
 4   species      150 non-null    category
dtypes: category(1), float64(4)
memory usage: 5.1 KB
```

0.10 The first four moments *of the iris statistical distribution*

The moments desribing the shape of the iris dataset distribution:

1) The mean (which indicates the central tendency of a distribution): varies across all species for each feature, except for Sepal measurements in setosa and versicolor, which are relatively similar
2) Variance (indicates the width or deviation): PetalWidth has narrow deviations from the mean for all species, while SepalWidth has a slightly wider variation for all species. Sepal and Petal length have much wider deviation

3) Skewness (indicates asymmetric 'leaning' to either left or right): Versicolor measurements are all negatively skewed, except for SepalLength, Virginica measurements are all positively skewed, except PetalWidth.
4) The Kurtosis, which indicates the degree of central 'peakedness' or, equivalently, the 'fatness' of the outer tails, indicates how much the distribution is influence by outliers. The distribution of PetalLength and PetalWidth are negatively influenced by outliers

```
metrics = ['count', 'min', 'max', 'mean','std','skew']
iris.groupby(by='species').agg(metrics)
```

	SepalLength						SepalWidth					
	count	min	max	mean	std	skew	count	min	max	mean	std	ske
species												
Iris-setosa	50	4.3	5.8	5.006	0.352490	0.120087	50	2.3	4.4	3.418	0.381024	0.1
Iris-versicolor	50	4.9	7.0	5.936	0.516171	0.105378	50	2.0	3.4	2.770	0.313798	-0.3
Iris-virginica	50	4.9	7.9	6.588	0.635880	0.118015	50	2.2	3.8	2.974	0.322497	0.3

```
iris.apply('kurtosis')
```

```
SepalLength   -0.552064
SepalWidth     0.290781
PetalLength   -1.401921
PetalWidth    -1.339754
dtype: float64
```

0

Exploratory data visualisation

Before modelling, we have to first of all ensure that there are no obvious groups that do not require clustering unnecessary sophisticated modelling to exploit to explain variations in our dataset.

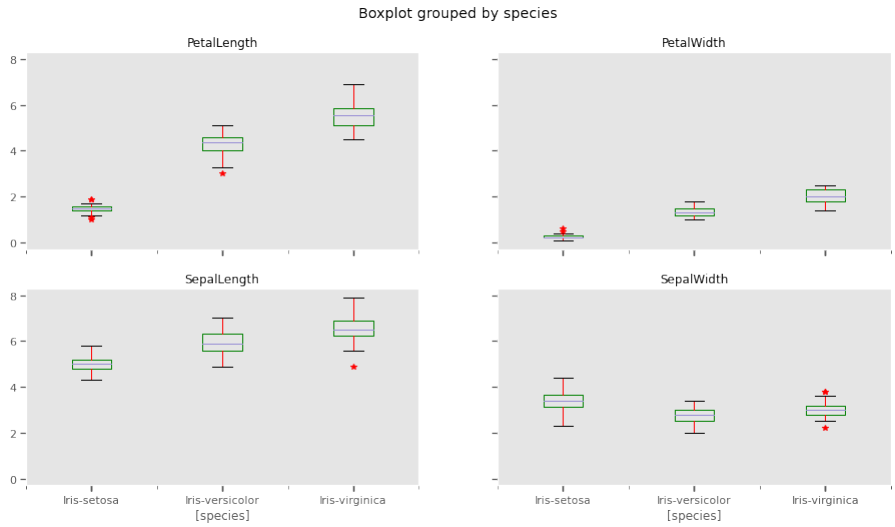
This is important in case our model introduces artefacts or does not achieve an optimum clustering outcome. Moreover, we would like to get a better understanding of our dataset to update our ground truth knowledge to develop better hypotheses we can test against our model outcomes. This is also important to address any confounding concerns for our decision to model relationships that might not be picked by our model or might be affected by our model.

0.10.1 Box plot

Based on the distribution of values within each column, sepal width and sepal length seem like weak discriminators for classification. At least 2 of the characteristics, petal length and petal width look like promising features to distinguish differerent species.

The whiskers extend from the edges of box to show the range of the data. The position of the whiskers is set by default to 1.5 * IQR (IQR = Q3 - Q1) from the edges of the box. Outlier points are those past the end of the whiskers. Outliers in some of the features need to be excluded from the dataset for models sensitive to outliers like k-means clustering

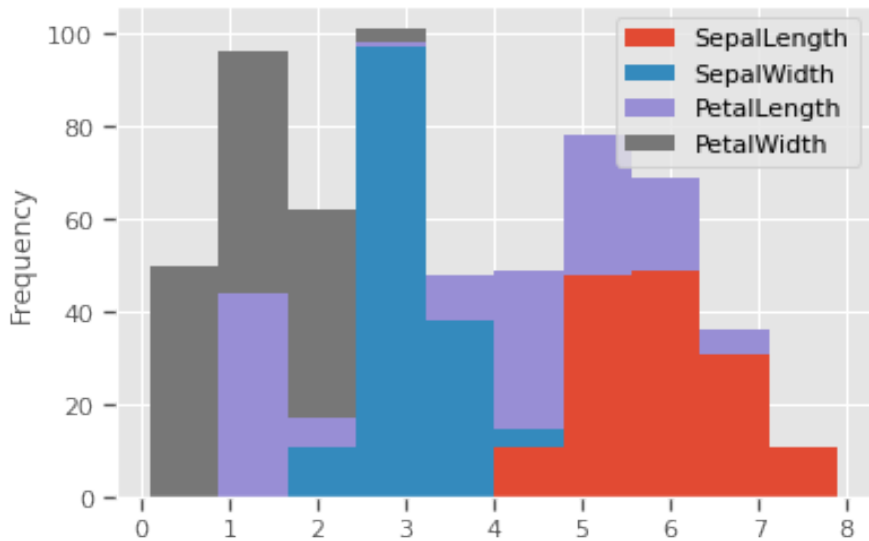
```
iris.boxplot(by='species', figsize=(15,
 ↪  8),color={'boxes':'Green','whiskers':'Red'},sym='r*',
 ↪  grid=False)
plt.show()
```



0.10.2 Histograms and Kernel Density estimation

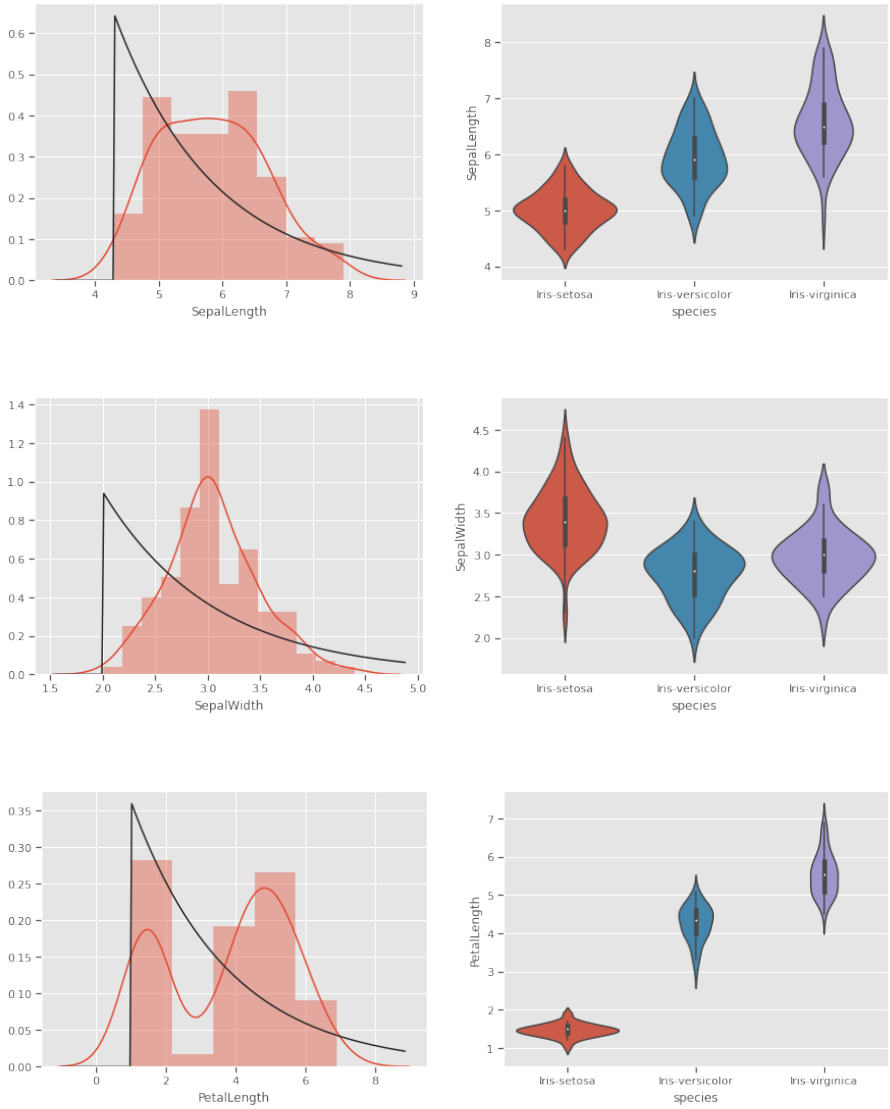
The scale of measurements for length are conflated, but not so much for width. This might affect clustering outcomes should we choose variables with similar measurements.

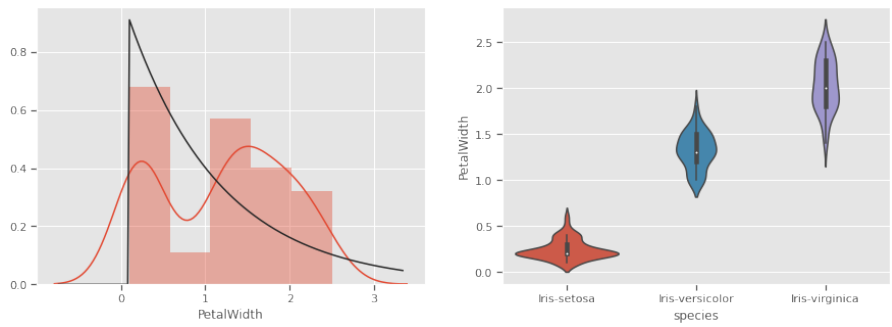
```
iris.plot.hist(stacked=True,by='species')
plt.show()
```



THe kernel density estimation (KDE) is a non-parametric way to estimate the probability density function (PDF) of a random variable. This function uses Gaussian kernels and includes automatic bandwidth determination. This plot estimates how normally distributed the features are. Petal width and petal length seem to be bi-normally distributed with some overlap, and could be weak discriminators

```
feats = iris.drop(columns=['species'])
for col in feats.columns:
  fig, (ax1,ax2) = plt.subplots(1, 2, figsize=(15,5),
↪  sharex=False)

↪  sns.distplot(iris[col],fit=stats.expon,hist=True,kde=True,ax=ax1)
  sns.violinplot(data=iris,x='species',y=col,inner='box',ax=ax2)
  plt.show()
```


SepalLength
SepalLength
species
Iris-setosa
Iris-versicolor
Iris-virginica
SepalWidth
SepalWidth
species
Iris-setosa
Iris-versicolor
Iris-virginica
PetalLength
PetalLength
species
Iris-setosa
Iris-versicolor
Iris-virginica



Petals measurements are bi-normally distributed and might make clustering using this feature as a discriminator

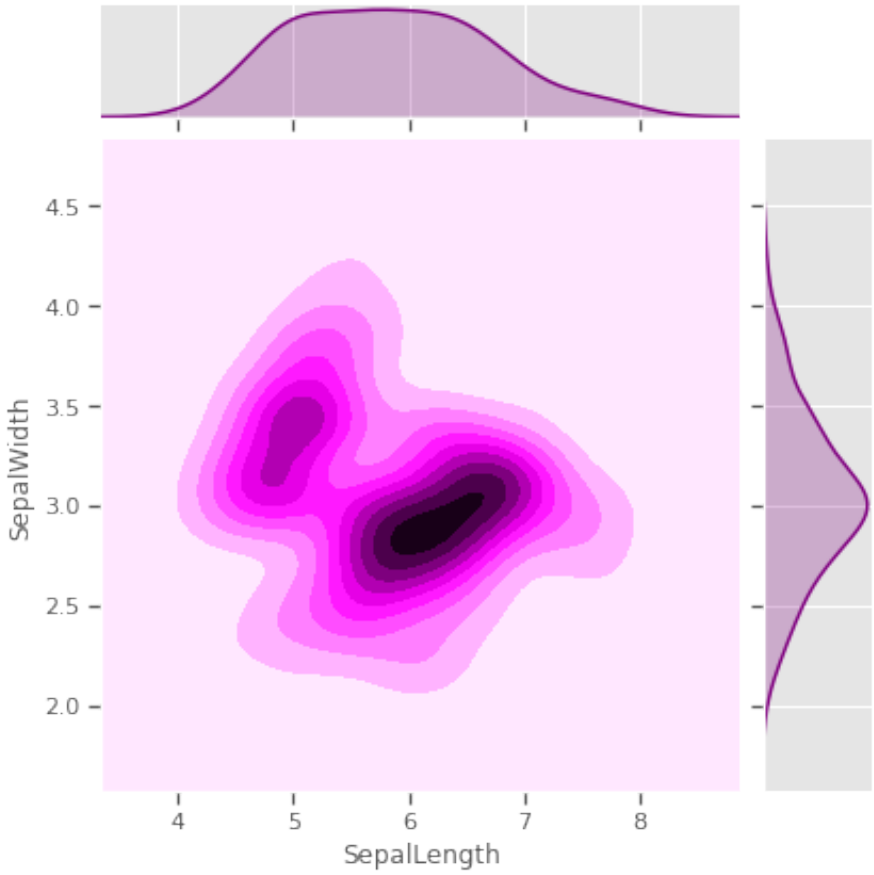
0.11 Multivariate analysis

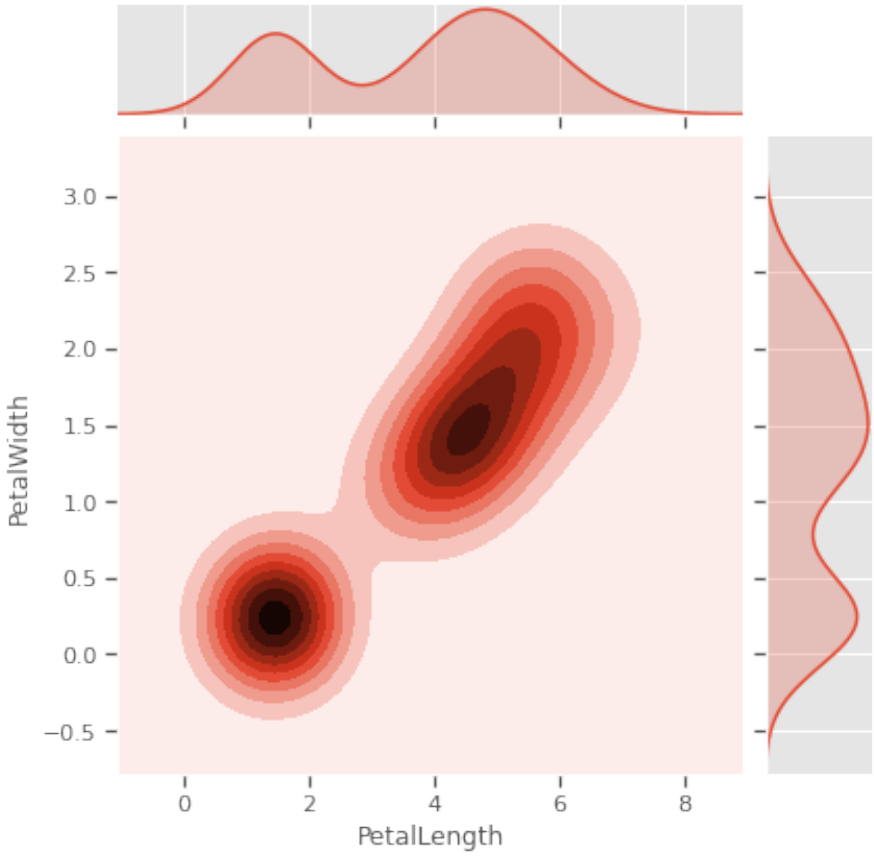
There is no confounding evidence of clusters forming based on univariate properties for any of the species. We will now explore multivariate interactions and the possibility of natural clusters forming. We would like the null hypothesis that "it is not possible to cluster all three iris species based on the Length and Width of Sepal and Petals"

0.11.1 Pairwise comparison

Sometimes variables measuring properties of the same entity have positive/negative re-enforcing effects, especially for biochemical systems like genetic phenotypical measurements. In our case Sepal and Petal properties might be genetically linked or depedent on each other in different species. Sepal measurements are seem like weak predictors of species, and will to cluster all three species. Petal measurements have a potential, but not strongly enough in pairwise relationships.

```
sns.jointplot(data=iris,x='SepalLength',y='SepalWidth',kind='kde',color="Purple")
sns.jointplot(data=iris,x='PetalLength',y='PetalWidth',kind='kde')
plt.savefig("kde.png")
plt.show()
```


4.5
4.0
3.5
3.0
2.5
2.0
SepalWidth
4
5
6
7
8
SepalLength



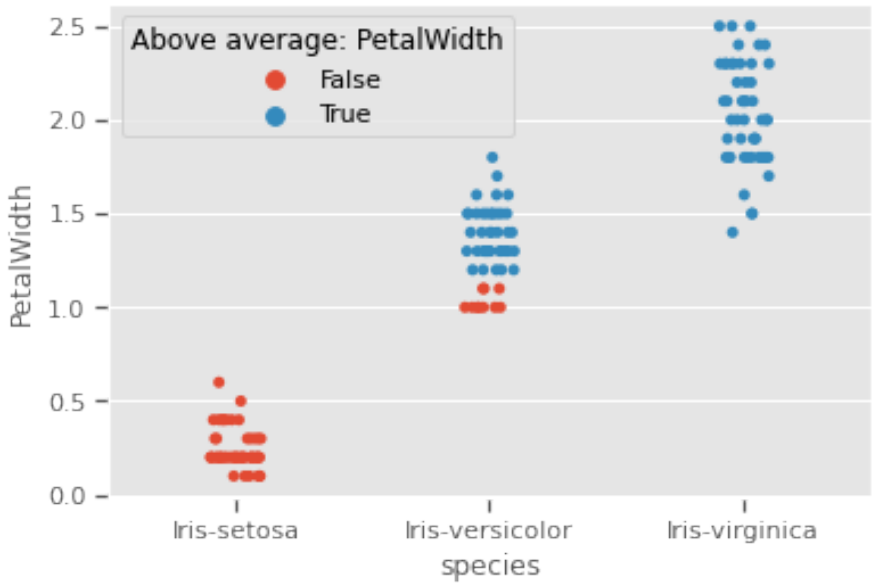
0.11.2 Scatterplot

We would like to test Sepal measurements against the species to see how much each species differs from the rest, and if skewness and kurtosis influencing natural formation of clusters. Species setosa has above average SepalWidth, but below average SepalLength, while versicolor is the exact opposite. Virginica has relatively above average SepalLength and SepalWidth.Sepal properties seem to have no generalisable species specific patterns. For example, just by looking at Species setosa and versicolor, one might be tempted to think that species with wider sepals have shorter sepals as well, but that is not the case for virginica. Prior knowledge of the genetics for these species confirms that

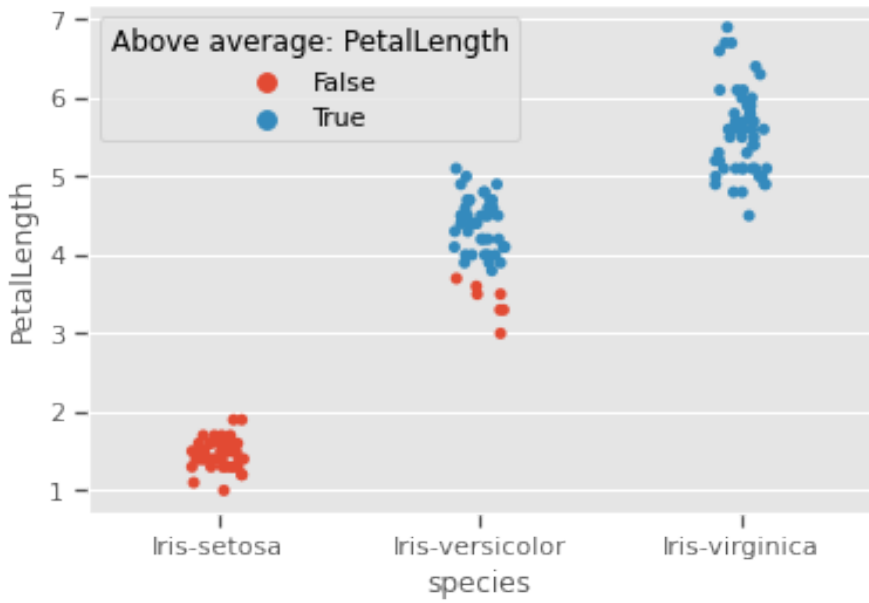
```
list_sepals = ("PetalWidth","PetalLength")
sepals = iris.copy()
```

```
for s in list_sepals:
  sepals[f'Above average: {s}'] = sepals[s] > sepals[s].mean()
  sns.stripplot(data=sepals,x='species',y=s,hue=f'Above average:
↪  {s}',jitter=True)
  print(f"Kurtosis:{round(sepals[s].apply('kurtosis'),2)}")
  plt.savefig("aboveavg_scatter.png")
  plt.show()
```

```
Kurtosis:-1.34
```



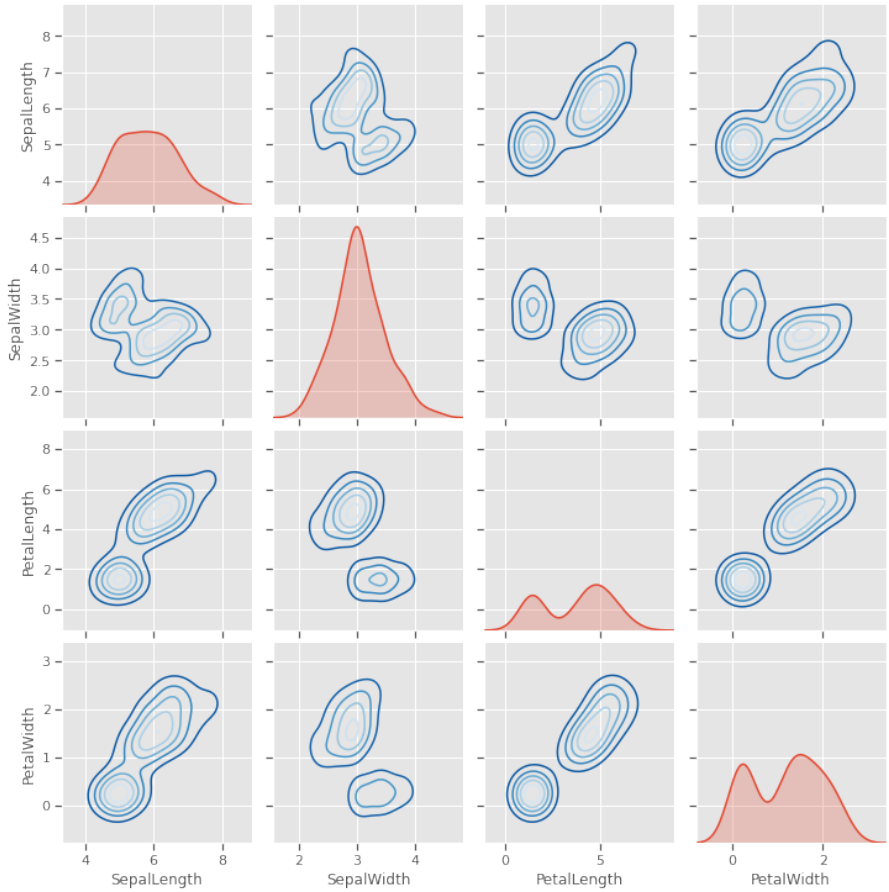

```
Kurtosis:-1.4
```



0.11.3 Pairgrid

Sepal length and sepal width seem to be useful as predictors for only 2 species, but not the third, as seen with all other variables.

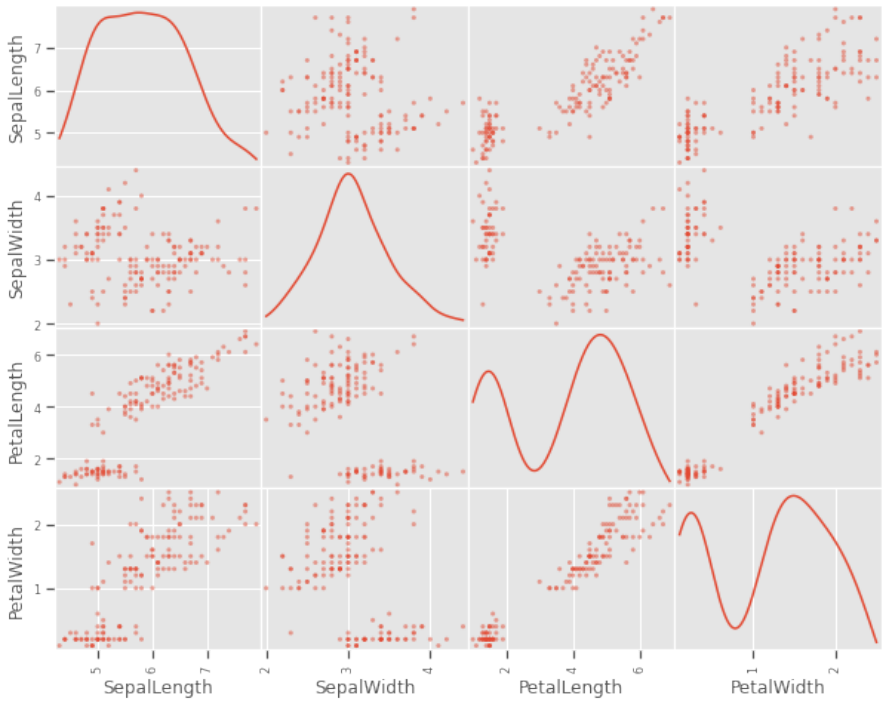
```
g = sns.PairGrid(iris)
g.map_diag(sns.kdeplot, shade=True)
g.map_offdiag(sns.kdeplot, cmap="Blues_r", n_levels=5)
plt.show()
```



0.11.4 Scatter matrix plot

A closer look at individual data points shows that at least one pair of features are discrimatory, but there is no evidence that two features will be sufficient to cluster the three species, some feature engineering might be needed. This will be investigated further in the visualisation below.

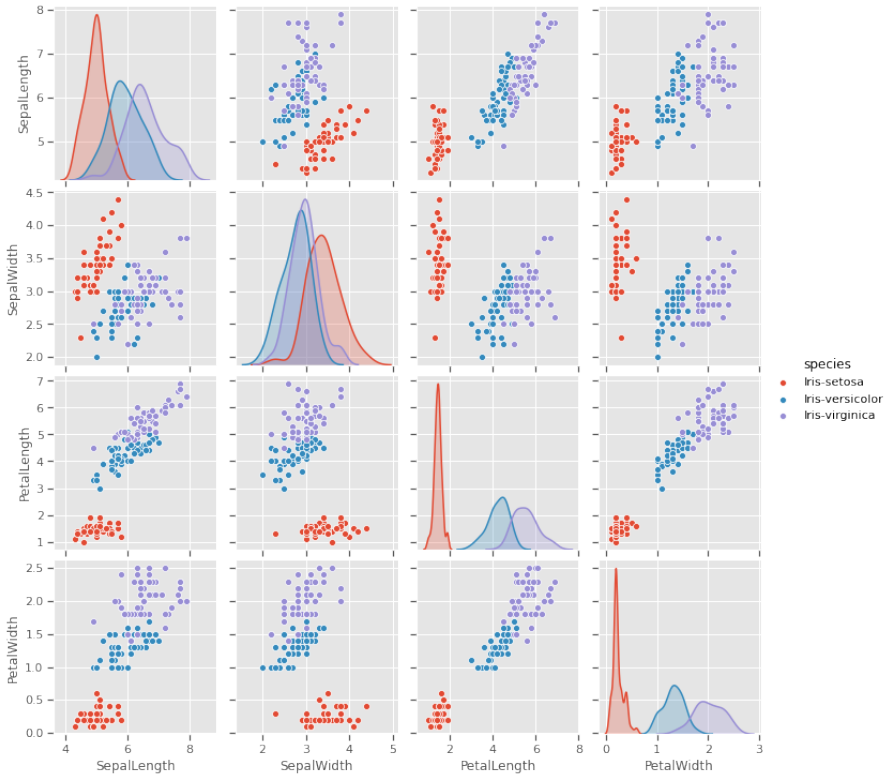
```
scatter_matrix(iris, figsize=(10,8), diagonal='kde')
plt.show()
```



0.11.5 Pairwise plot

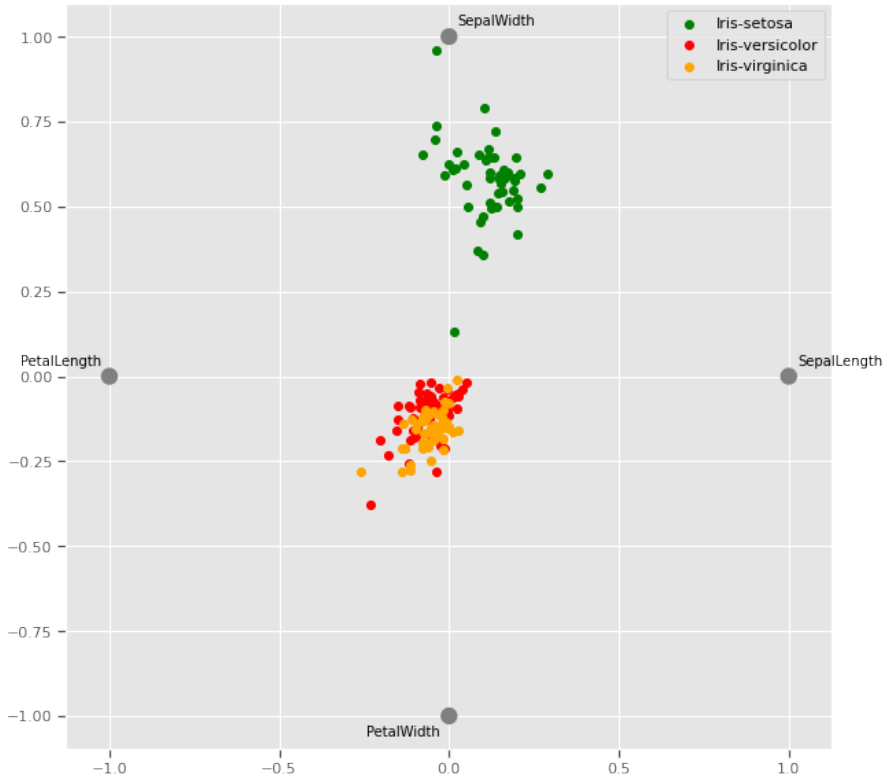
A closer look at the species variations shows that there is a good chance to cluster sepal and petal measurements into the three species, but versicolor and virginica species are showing very close resemblance on all variables. This might be challenging for a naive k-means clustering process to optimally cluster these two species

```
sns.pairplot(iris,hue='species')
plt.savefig("pairwise.png")
plt.show()
```



0.11.6 Radviz plot

```
plt.figure(figsize=(10, 10))
radviz(iris, 'species',color=['Green','Red','Orange'])
plt.savefig("radviz.png")
plt.show()
```



The Radviz plot helps visualise multi-variate data based on a simple spring tension minimization algorithm. The set up places points in a plane equally spaced on a unit circle. Each point represents a single attribute. Each sample in the data set is abstractly "attached" to each of these points by a spring, the stiffness of which is proportional to the numerical value of that attribute (they are normalized to unit interval). The point in the plane, where our sample settles to (where the forces acting on our sample are at an equilibrium) is where a dot representing our sample will be drawn. Depending on which class that sample belongs it will be colored differently.

Virginia and versicolor species seem to have closely matched feature measurements, and might require a third dimension to cluster.

0.11.7 Andrews curves

Andrews curves are useful in visualizing multivariate data as a large number of curves that are created using the attributes of samples as coefficients for Fourier series. By coloring these curves differently for each class it is possible

to visualize data clustering. Curves belonging to samples of the same class will usually be closer together and form larger structures.

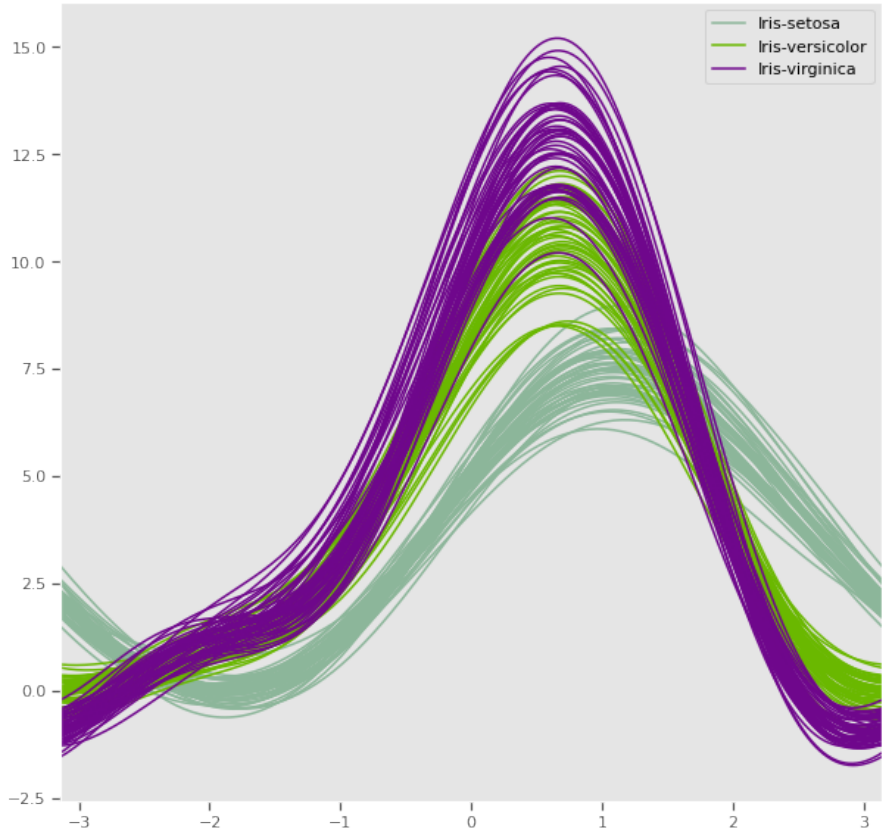
Andrews curves have the functional form of a Fourier series:

```
f(t) = x_1/sqrt(2) + x_2 sin(t) + x_3 cos(t) + x_4 sin(2t) + x_5 cos(2t) + ...
```

Where x coefficients correspond to the values of each dimension and t is linearly spaced between -pi and +pi. Each row of frame then corresponds to a single curve.

The iris dataset has 2 distinct classes, but the third class is visibly related to one of the other two classes and will require a mathematical model to optimally cluster.

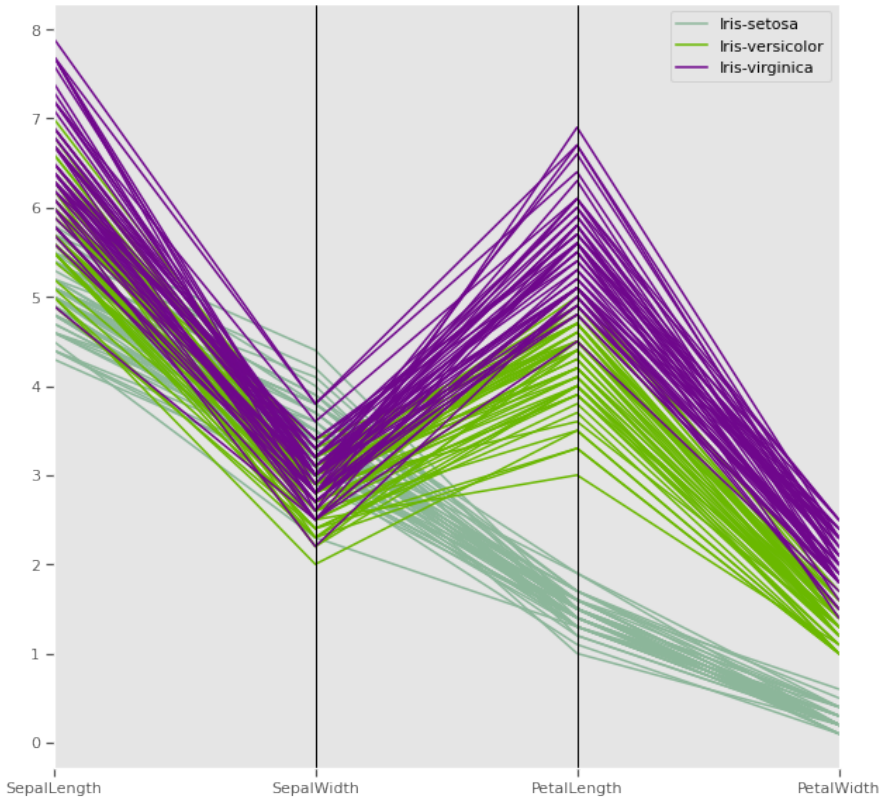
```
plt.figure(figsize=(10, 10))
andrews_curves(iris, 'species')
plt.savefig("andrew_curve.png")
plt.show()
```



0.11.8 Parallel coordinates

Petal length and petal width has a very strong discriminatory power, while sepal measurements might be very useful together, one might be enough to improve cluster distance

```
plt.figure(figsize=(10, 10))
parallel_coordinates(iris, 'species')
plt.show()
```



Parallel coordinates help with visualizing clusters in data and to estimate other statistics visually. Using parallel coordinates points are represented as connected line segments. Each vertical line represents one attribute. One set of connected line segments represents one data point. Points that tend to cluster will appear closer together.

In the iris dataset, Petal length has potential have high discriminatory power for clustering compared to the other three attributes

0.11.9 Ground truth - Three dimenstional plot

Taking advantage of prior knowlegde about the iris dataset regarding the species, we can visualise the natural clusters before modelling, as we previously observed in multivariate analysis, at least three variables are sufficient to cluster our flowers into the three different species

```
data = iris.copy()
X = data.drop(columns=['species'])
y = iris['species'] # k-means does not handle categories, unless
 ↪  labelencoded or dummy variables are created.
y.unique()
```

```
[Iris-setosa, Iris-versicolor, Iris-virginica]
Categories (3, object): [Iris-setosa, Iris-versicolor, Iris-virginica]
```

```
!pip install orca psutil requests
```

```
Requirement already satisfied: orca in /usr/local/lib/python3.6/dist-packages (1.5.3)
Requirement already satisfied: psutil in /usr/local/lib/python3.6/dist-packages (5.4.8)
Requirement already satisfied: requests in /usr/local/lib/python3.6/dist-packages (2.23.0)
Requirement already satisfied: toolz>=0.8.1 in /usr/local/lib/python3.6/dist-packages (from orca) (0.
Requirement already satisfied: pandas>=0.15.0 in /usr/local/lib/python3.6/dist-packages (from orca) (
Requirement already satisfied: tables<3.7,>=3.1; python_version >= "3.6" in /usr/local/lib/python3.6/
Requirement already satisfied: urllib3!=1.25.0,!=1.25.1,<1.26,>=1.21.1 in /usr/local/lib/python3.6/d
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.6/dist-packages (from req
Requirement already satisfied: idna<3,>=2.5 in /usr/local/lib/python3.6/dist-packages (from requests)
Requirement already satisfied: chardet<4,>=3.0.2 in /usr/local/lib/python3.6/dist-packages (from requ
Requirement already satisfied: pytz>=2017.2 in /usr/local/lib/python3.6/dist-packages (from pandas>=0
Requirement already satisfied: numpy>=1.13.3 in /usr/local/lib/python3.6/dist-packages (from pandas>=
Requirement already satisfied: python-dateutil>=2.6.1 in /usr/local/lib/python3.6/dist-packages (fro
Requirement already satisfied: numexpr>=2.5.2 in /usr/local/lib/python3.6/dist-packages (from tables<
Requirement already satisfied: six>=1.9.0 in /usr/local/lib/python3.6/dist-packages (from tables<3.7,
```

```
import plotly.express as px
import plotly
fig = px.scatter_3d(iris, x='PetalLength', y='SepalWidth',
 ↪  z='PetalWidth',color='species',title="Iris dataset ground
 ↪  truth")
# fig.write_image("iris_3d.png")
fig.show()
```

```
Unable to display output for mime type(s): text/html
```

0

K-means clustering

The k-means problem is solved using either Lloyd's or Elkan's algorithm, a relatively fast algorithm. The average complexity is given by O(k n T): - n is the number of samples - T is the number of iterations The worst case complexity is given by O(n^(k+2/p)) with n = n_samples, p = n_features. (D. Arthur and S. Vassilvitskii, 'How slow is the k-means method?' SoCG2006)

Adapted from official documentation for sklearn[15]

As with most statistical algorithms, there are some assumptions, which, if not met, the algorithm will fail dismally.

For k-means clustering: - Assumption 1: each group will belong to not more than one cluster and that the clusters are of similar/comparable sizes (choose optimum number of clusters) - Assumption 2: clusters are spherical and not dependent on direction (not anisotropic) i.e deal with outliers or normalise data

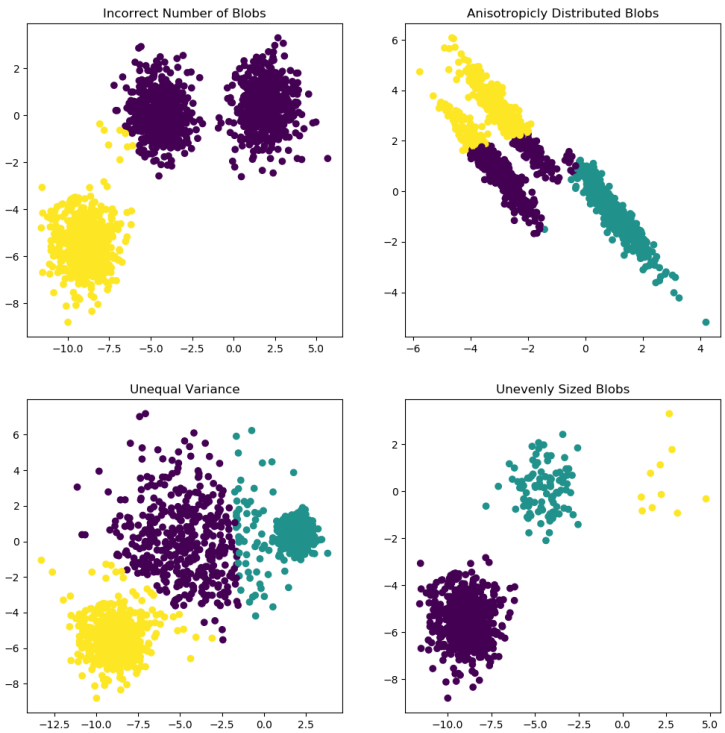
The K-means algorithm aims to choose centroids that *minimise the inertia, or within-cluster sum-of-squares* criterion:

Inertia can be recognized as a measure of how internally coherent clusters are. It suffers from various drawbacks: - Inertia makes the assumption that clusters are convex and isotropic, which is not always the case. It responds poorly to elongated clusters, or manifolds with irregular shapes.

Inertia is not a normalized metric: we just know that lower values are better and zero is optimal. But in very high-dimensional spaces, Euclidean distances tend to become inflated (this is an instance of the so-called "curse of dimensionality"). Running a dimensionality reduction algorithm such as Principal component analysis (PCA) prior to k-means clustering can alleviate this problem and speed up the computations.

```
from IPython.display import Image, display
image =
 ↪  'https://scikit-learn.org/stable/_images/sphx_glr_plot_kmeans_assumptions_001.png'
display(Image(image,width=700,height=700))
```

[15] http://scikit-learn.org/stable/auto_examples/svm/plot_iris.html


Incorrect Number of Blobs
Anisotropicly Distributed Blobs
Unequal Variance
Unevenly Sized Blobs

0

Optimisation

- Elbow method
- Silhoutte method

0.11.10 Sihlouette method

A higher Silhouette Coefficient indicates that the object is well matched to its own cluster and poorly matched to neighboring clusters.

Sihlouette outcome: del n_clusters=2: highest Silhouette Coefficient. This suggests that 2 should be the optimal number of cluster

Ground truth: Iris dataset has 3 species, this contradicts the 2 as an optimal number of cluster. So despite n_clusters=2 having highest Silhouette Coefficient

We would consider n_clusters=3 as optimal number of clusters due to: - Iris dataset has 3 species - n_clusters=2 has a 2nd highest value of Silhouette Coefficient.

So choosing n_clusters=3 is the optimal no. of cluster for iris dataset. This is where EDA and having full context of the dataset really matters.

```
from sklearn.metrics import silhouette_score
for n_cluster in range(2, 11):
    kmeans = KMeans(n_clusters=n_cluster).fit(X)
    label = kmeans.labels_
    sil_coeff = silhouette_score(X, label, metric='euclidean')
    print(f"{n_cluster} n_clusters: Silhouette Coefficient =
     ↪  {sil_coeff}")
```

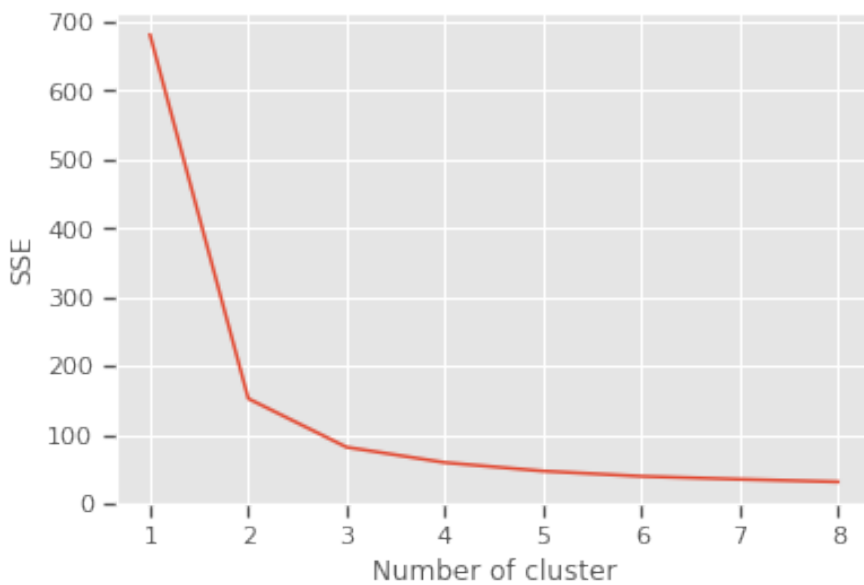
```
2 n_clusters: Silhouette Coefficient = 0.5913935910002965
3 n_clusters: Silhouette Coefficient = 0.7044451503494762
4 n_clusters: Silhouette Coefficient = 0.6434117378803829
5 n_clusters: Silhouette Coefficient = 0.6420432882616431
6 n_clusters: Silhouette Coefficient = 0.5939145825544134
7 n_clusters: Silhouette Coefficient = 0.5607633412528588
```

```
8 n_clusters: Silhouette Coefficient = 0.5880832106447507
9 n_clusters: Silhouette Coefficient = 0.5473372266420964
10 n_clusters: Silhouette Coefficient = 0.494472198641177
```

0.11.11 Elbow method

The Elbow Method is a method to determine the optimal value of k (number of centroids)

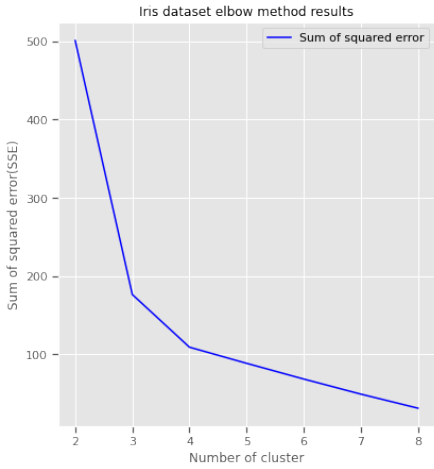
```
sse = {}
for k in range(1,9):
    kmeans = KMeans(n_clusters=k, max_iter=1000).fit(X)
    X["clusters"] = kmeans.labels_
    sse[k] = kmeans.inertia_ # Inertia: Sum of distances of
 ↪  samples to their closest cluster center
plt.figure()
plt.plot(list(sse.keys()), list(sse.values()))
plt.xlabel("Number of cluster")
plt.ylabel("SSE")
plt.show()
```

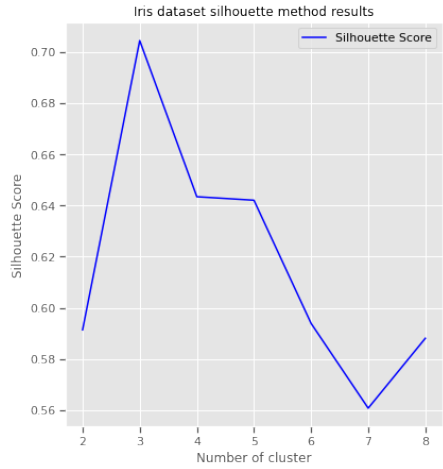


Silhouette and Elbow method subplots

```
range_n_clusters = range(2,9)
elbow = []
ss = []
for n_clusters in range_n_clusters:
   #iterating through cluster sizes
   clusterer = KMeans(n_clusters = n_clusters, random_state=42)
   cluster_labels = clusterer.fit_predict(X)
   #Finding the average silhouette score
   silhouette_avg = silhouette_score(X, cluster_labels)
   ss.append(silhouette_avg)
   print(f"For {n_clusters} clusters: average silhouette_score
    ↪  is {silhouette_avg}")
   #Finding the average SSE"
   elbow.append(clusterer.inertia_) # Inertia: Sum of distances
 ↪  of samples to their closest cluster center

fig = plt.figure(figsize=(14,7))

fig.add_subplot(121)
plt.title("Iris dataset elbow method results")
plt.plot(range_n_clusters, elbow,'b-',label='Sum of squared
 ↪  error')
plt.xlabel("Number of cluster")
plt.ylabel("Sum of squared error(SSE)")
plt.legend()

fig.add_subplot(122)
plt.title("Iris dataset silhouette method results")
plt.plot(range_n_clusters, ss,'b-',label='Silhouette Score')
plt.xlabel("Number of cluster")
plt.ylabel("Silhouette Score")
plt.legend()
plt.show()
```

```
For 2 clusters: average silhouette_score is 0.5913935910002965
For 3 clusters: average silhouette_score is 0.7044451503494762
For 4 clusters: average silhouette_score is 0.6434117378803829
For 5 clusters: average silhouette_score is 0.6420432882616431
For 6 clusters: average silhouette_score is 0.5939145825544134
For 7 clusters: average silhouette_score is 0.5607633412528588
For 8 clusters: average silhouette_score is 0.5880832106447507
```


Iris dataset elbow method results
Sum of squared error
Sum of squared error(SSE)
500
400
300
200
100
2
3
4
5
6
7
8
Number of cluster


Iris dataset silhouette method results
Silhouette Score
Silhouette Score
0.70
0.68
0.66
0.64
0.62
0.60
0.58
0.56
2
3
4
5
6
7
8
Number of cluster

0

Train model

- *n_init* Number of times algorithm will be run with different centroid seeds.Results will be best inertia output. Setting n_init to 1 (default is 10), reduces the amount of times algorithm will be run with different centroid seeds is reduced.
- *n_clusters*: based on Elbow and Silhouette method results
- *algorithm*: "elkan" or "auto" or "full". The classical EM-style (Lloyd's) algorithm is "full". The "elkan" variation is more efficient by using the triangle inequality, but currently doesn't support sparse data. "auto" chooses "elkan" for dense data and "full" for sparse data.
- *max_iter*: the k-means algorithm is generally very fast, but it falls on a local minima. That's why it can be useful to restart it several times. If the algorithm stops before fully converging (because of tol or max_iter), labels_ and cluster_centers_ will not be consistent, i.e. the cluster_centers_ will not be the means of the points in each cluster. Also, the estimator will reassign labels_ after the last iteration to make labels_ consistent with predict on the training set.

```
from sklearn.cluster import KMeans
model = KMeans(n_clusters=3,n_init=1,init='random',n_jobs = 4,
↪  max_iter=10000, random_state=21,algorithm="auto")
fitted_model = model.fit(X)
labels = model.labels_
centers = pd.DataFrame(fitted_model.cluster_centers_)
print(f'Cluster centers: \n {centers}')
```

```
Cluster centers:
           0         1         2         3         4
0  5.006000  3.418000  1.464000  0.244000  2.520000
1  6.472881  2.930508  5.345763  1.920339  5.220339
2  5.958537  2.787805  4.273171  1.324390  0.512195
```

0

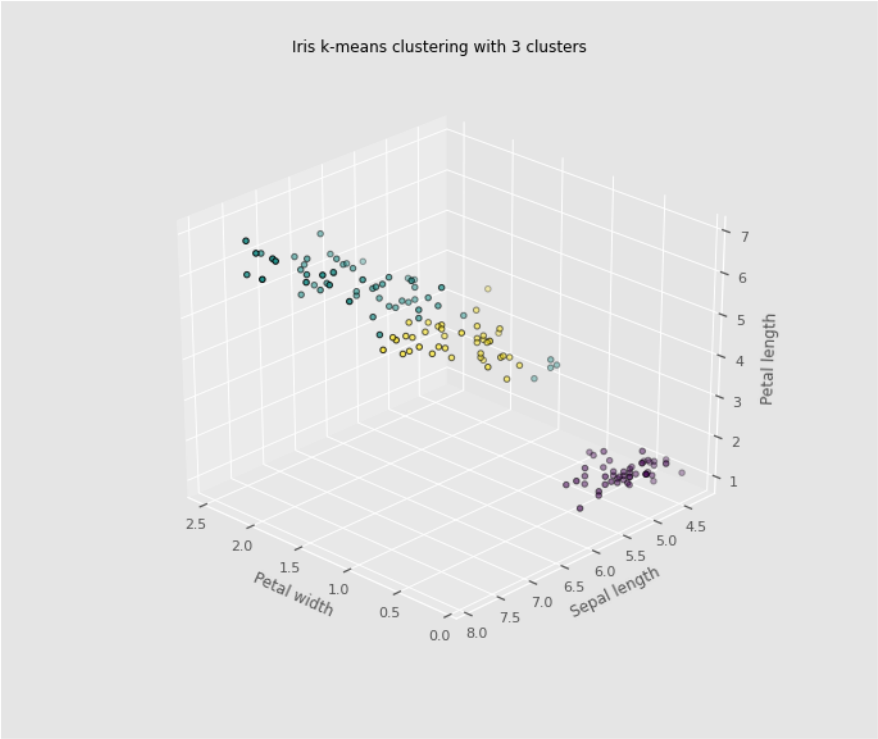
Visualise predictions: k-means

The iris dataset is a great dataset to demonstrate some of the shortcomings of k-means clustering. This dataset also presents a great opportunity to highlight the importance of exploratory data analysis to understand the data and gain more insights about the data before deciding which clustering algorithm to use and whether or a model is necessary to group the data.

Without prior knowledge of the species of iris flowers in this dataset, multivariate analysis was proven to be an ineffective way to cluster the iris dataset. Clustering seemed like a good candidate for grouping the iris dataset.However, the results in the case of the iris dataset show that two out of the three species are difficult to cluster because their clusters are not spherical and are elongated (anisotropic) in the possible direction clusters can form. Based on the ground truth evidence, and prior knowledge of the species, from the 3D plot, it is evidenced that k-means cluster was not able to improve clustering of the three iris species in the dataset. A dimensionality reduction using Principal Component analysis might improve the efficiency of the k-means clustering algorithm, used independently, it will not produce good enough clusters.

```
from mpl_toolkits.mplot3d import Axes3D

fig = plt.figure(figsize=(10, 8))
ax = Axes3D(fig,
            rect=[0, 0, .95, 1],
            elev=30,
            azim=134)
model.fit(X)
labels = model.labels_

ax.set_title("Iris k-means clustering with 3 clusters")
ax.scatter(X["PetalWidth"], X["SepalLength"],
 ↪  X["PetalLength"],c=labels.astype(np.float), edgecolor='k')
ax.set_xlabel('Petal width',labelpad=10)
ax.set_ylabel('Sepal length',labelpad=10)
ax.set_zlabel('Petal length',labelpad=10)
```

```
ax.dist = 13
plt.savefig("kclusters.png")
```



0

Contoh

$$\chi^2 = \sum_{i,j} \frac{\left(O_{ij} - E_{ij}\right)^2}{E_{ij}}$$

makanan itu dapat digunakan dengan $x + 2$

```
pip install academics-scholar-scraper
```

```
Collecting academics-scholar-scraper
  Downloading academics_scholar_scraper-0.1.1-py3-none-any.whl (5.5 kB)
Collecting argparse (from academics-scholar-scraper)
  Downloading argparse-1.4.0-py2.py3-none-any.whl (23 kB)
Requirement already satisfied: requests in /usr/local/lib/python3.10/dist-packages (from academics-sc
Requirement already satisfied: tqdm in /usr/local/lib/python3.10/dist-packages (from academics-schola
Collecting openai (from academics-scholar-scraper)
  Downloading openai-0.27.8-py3-none-any.whl (73 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 73.6/73.6 kB 3.5 MB/s eta 0:00:00
Requirement already satisfied: aiohttp in /usr/local/lib/python3.10/dist-packages (from openai->acade
Requirement already satisfied: urllib3<1.27,>=1.21.1 in /usr/local/lib/python3.10/dist-packages (fro
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.10/dist-packages (from re
Requirement already satisfied: charset-normalizer~=2.0.0 in /usr/local/lib/python3.10/dist-packages
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.10/dist-packages (from requests
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.10/dist-packages (from aiohttp
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.10/dist-packages (from a
Requirement already satisfied: async-timeout<5.0,>=4.0.0a3 in /usr/local/lib/python3.10/dist-package
Requirement already satisfied: yarl<2.0,>=1.0 in /usr/local/lib/python3.10/dist-packages (from aiohtt
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.10/dist-packages (from aio
Requirement already satisfied: aiosignal>=1.1.2 in /usr/local/lib/python3.10/dist-packages (from aioh
Installing collected packages: argparse, openai, academics-scholar-scraper
Successfully installed academics-scholar-scraper-0.1.1 argparse-1.4.0 openai-0.27.8
```

```
Unable to display output for mime type(s): application/vnd.colab-display-data+json
```

```
pip install argparse requests tqdm concurrent.futures openai
```

```
Collecting argparse
  Using cached argparse-1.4.0-py2.py3-none-any.whl (23 kB)
Requirement already satisfied: requests in /usr/local/lib/python3.10/dist-packages (2.27.1)
Requirement already satisfied: tqdm in /usr/local/lib/python3.10/dist-packages (4.65.0)
ERROR: Could not find a version that satisfies the requirement concurrent.futures (from versions: none)
ERROR: No matching distribution found for concurrent.futures
```

0

Summary

In summary, this book has no content whatsoever.

References